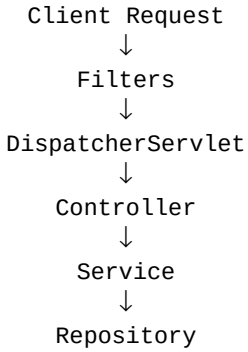
Spring Boot Filters

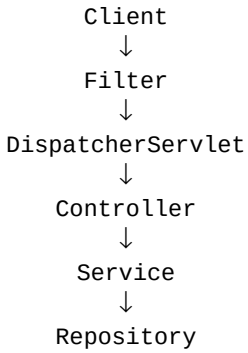
Lecture notes • Java examples • Top-to-bottom text diagrams • Interview revision



1. Why Do We Need Filters?

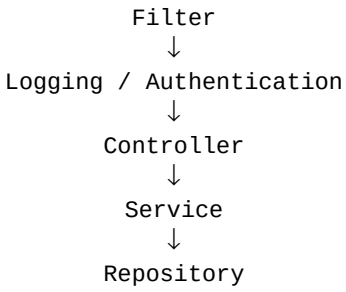
- Applications often need common operations before or after multiple API endpoints.
- Examples: request/response logging, authentication, request ID generation, response headers, and measuring processing time.
- Putting this logic in every controller causes repeated code and tight coupling.
- A separate helper class still requires controllers to call it manually.

Solution: Use Filters



2. Filter and Single Responsibility Principle (SRP)

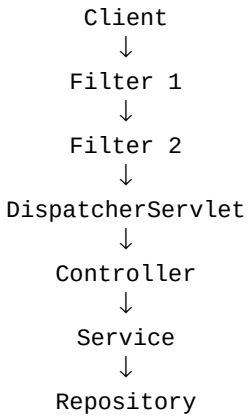
- **Filter:** common request/response processing.
- **Controller:** receives requests and delegates to the service.
- **Service:** executes business logic.
- **Repository:** handles data access.



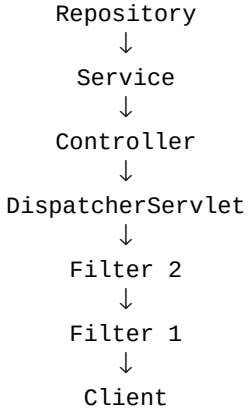
Key point: Logging and authentication are infrastructure/security concerns, not the controller's primary responsibility.

3. Where Does a Filter Fit?

Incoming request flow



Outgoing response flow



- Filters execute before DispatcherServlet processes the incoming request.
- The response passes back through the filter chain before reaching the client.
- At filter time, Spring MVC has not yet selected the controller handler.
- Filters may also encounter certain static-resource or error requests that never reach a controller.

4. Filter vs Interceptor

Filter	Interceptor
Part of Servlet API	Part of Spring MVC
Runs before DispatcherServlet	Runs inside Spring MVC
Before controller selection	Closer to handler execution
General request/response processing	Handler-related processing

5. Common Uses of Filters

- **Logging:** inspect method, URI, headers and response status; avoid sensitive data.
- **Authentication:** check credentials and reject invalid requests.
- **Response modification:** add headers such as a request ID.
- **Timing:** measure downstream request processing time.
- **Other uses:** inspect cookies, apply wrappers or encoding-related processing, and block/allow requests.

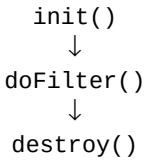
6. Servlet Filter Interface and Lifecycle

The Filter interface comes from the Servlet API, not directly from Spring.

```
jakarta.servlet.Filter
```

Three lifecycle methods

Method	Purpose
init()	Initializes the filter.
doFilter()	Main per-request method; processes the request and continues or stops the chain.
destroy()	Performs cleanup when the filter is taken out of service.



Basic filter structure

```
@Component
public class LoggingFilter implements Filter {
@Override
public void doFilter(ServletRequest request,
ServletResponse response,
FilterChain chain)
throws IOException, ServletException {
// Filter logic
}
}
```

Implement the Servlet API Filter interface and define doFilter().

7. Understanding doFilter() Parameters

```
doFilter(
ServletRequest request,
ServletResponse response,
FilterChain chain
)
```

Request and response

- **ServletRequest / ServletResponse:** generic Servlet API types.
- For HTTP-specific methods, cast them to HttpServletRequest and HttpServletResponse.
- HTTP request provides the method, URI, headers, parameters and body.
- HTTP response sets status, headers, content type and body.

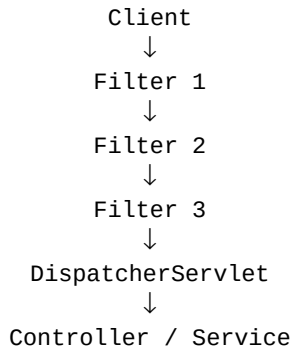
```
HttpServletRequest httpRequest =
(HttpServletRequest) request;

HttpServletResponse httpResponse =
(HttpServletResponse) response;
```

FilterChain

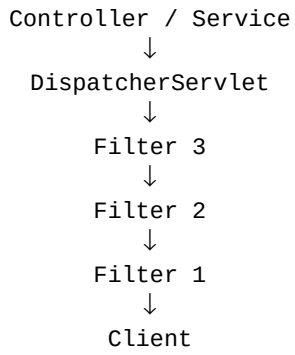
- chain.doFilter(request, response) passes processing to the next filter.
- If no filters remain, processing continues to the target servlet.
- Code before the call runs on the incoming path; code after it runs when downstream processing returns.
- Modern Spring Boot uses jakarta.servlet.*; older versions may use javax.servlet.*.

8. Understanding the Filter Chain

Request flow



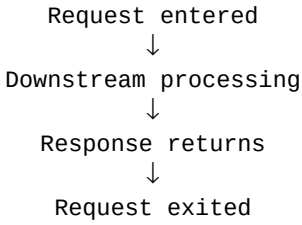
Response flow



Example

```
System.out.println("Request entered");

chain.doFilter(request, response);

System.out.println("Request exited");
```

Expected order



- Each filter performs its work and calls chain.doFilter() to continue.
- If a filter does not continue the chain, downstream processing does not run.

9. Creating a Logging Filter

Log the HTTP method and URI before processing, then log the response status afterwards.

```
@Component
public class LoggingFilter implements Filter {
@Override
public void doFilter(ServletRequest request,
ServletResponse response,
FilterChain chain)
throws IOException, ServletException {
HttpServletRequest req =
(HttpServletRequest) request;
HttpServletResponse res =
(HttpServletResponse) response;

System.out.println("Incoming request: "
+ req.getMethod() + " " + req.getRequestURI());

try {
chain.doFilter(request, response);
} finally {
System.out.println("Response status: "
+ res.getStatus());
}
}
}
```

What is logged?

- req.getMethod() — HTTP method.
- req.getRequestURI() — requested path.
- res.getStatus() — response status code.

Example output

```
Incoming request: POST /api/students
Student created: Aditya
Response status: 200
```

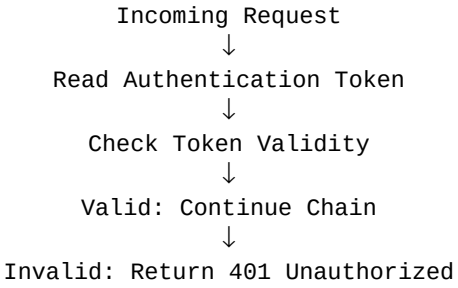
Use a logging framework in real applications; console printing is for demonstration.

10. Authentication Filter

The lecture demonstrates a simplified token check. This is an educational example, not production-grade authentication.

Assume the expected token is 12345.

```
String token = httpRequest.getHeader("token");

if (!"12345".equals(token)) {
httpResponse.setStatus(
HttpServletResponse.SC_UNAUTHORIZED
);
return;
}

chain.doFilter(request, response);
```



Important points

- getHeader("token") reads the token from the request header.
- Missing or invalid token: return 401 Unauthorized.
- Valid token: call chain.doFilter() to continue.
- Real applications should use proper token validation and avoid hardcoded production credentials.

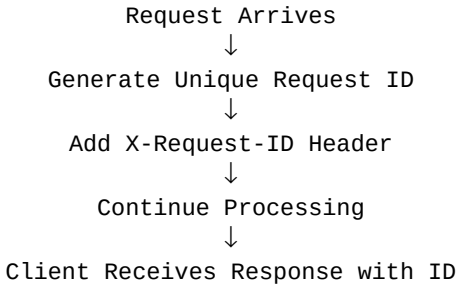
11. Adding a Request ID to the Response

A unique request ID helps correlate a client request with server-side logs.

```
String requestId = UUID.randomUUID().toString();

httpResponse.setHeader(
"X-Request-ID",
requestId
);

chain.doFilter(request, response);
```

Required import

```
import java.util.UUID;
```



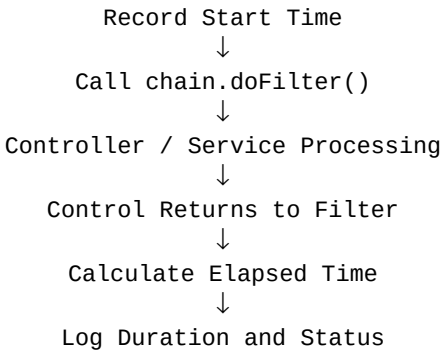
Why useful?

- Helps identify a specific request.
- Makes troubleshooting and log searches easier.
- Useful when integrating with third-party services.
- Set headers before the response is committed.

12. Measuring API Response Time

Formula: Duration = End Time – Start Time

```
long startTime = System.currentTimeMillis();

try {
chain.doFilter(request, response);
} finally {
long duration =
System.currentTimeMillis() - startTime;

System.out.println(
"API response time: " + duration + " ms"
);

System.out.println(
"Response status: " + httpResponse.getStatus()
);
}
```

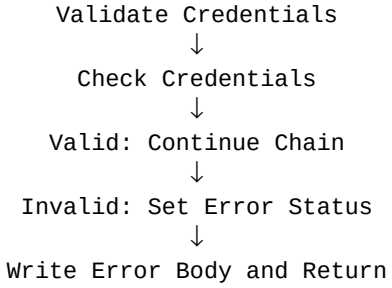


- Measured time includes downstream processing performed during the call.
- finally allows timing/status logging to run even if downstream execution throws an exception.
- A simulated delay is for demonstration only, not production code.

13. Returning an Error Response from a Filter

A filter can reject a request before it reaches the controller and return an error response directly.

```
httpResponse.setStatus(
HttpServletResponse.SC_UNAUTHORIZED
);

httpResponse.setContentType("application/json");
httpResponse.getWriter().write(
"{\"message\":\"Invalid or missing API key\"}"
);
return;
```



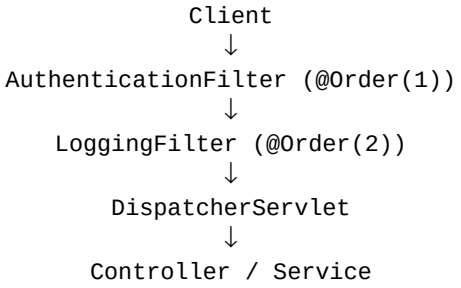
Remember

- Set the appropriate HTTP status and content type.
- Write the error response body, then return.
- Do not call chain.doFilter() when the request must be blocked.
- The lecture uses 401 Unauthorized for invalid or missing authentication information.

14. Ordering Multiple Filters with @Order

Use separate filters for separate responsibilities and define their execution order.

```
@Component
@Order(1)
public class AuthenticationFilter implements Filter {
// Authentication logic
}

@Component
@Order(2)
public class LoggingFilter implements Filter {
// Logging logic
}
```



Key points

- In this example, authentication runs before logging.
- Explicit ordering helps make execution predictable.
- Response processing returns through the filters in reverse order.
- Keep each filter focused on one responsibility.

15. Important Concepts — Quick Comparison

Concept	Purpose
Filter	Processes requests and responses.
doFilter()	Executes filter logic.
FilterChain	Continues processing to the next filter.
HttpServletRequest	Provides HTTP request information.
HttpServletResponse	Controls the HTTP response.
@Component	Registers the filter as a Spring-managed component.
@Order	Defines filter execution order.

16. Best Practices

- Keep logging and authentication logic out of controllers.
- Use separate filters for separate responsibilities.
- Call chain.doFilter() when processing should continue.
- Use try/finally for required post-processing.
- Set response headers before the response is committed.
- Avoid logging sensitive information or hardcoding production secrets.
- Avoid modifying the original request unnecessarily.

17. Interview Questions and Answers

What is a Servlet Filter?

A Servlet API component that can inspect or process requests and responses.

Is Filter a Spring-specific concept?

No. It is provided by the Servlet API and can be used in Spring Boot.

Where does a filter execute?

Before DispatcherServlet on the incoming request path.

What is doFilter()?

The main method where per-request filter logic is implemented.

What is FilterChain?

It passes control to the next filter or the target servlet.

What happens if chain.doFilter() is not called?

Downstream processing does not continue through that chain.

Why use finally?

To run post-processing, such as duration/status logging, even if an exception occurs.

How do you order filters?

Use @Order to specify intended execution order.

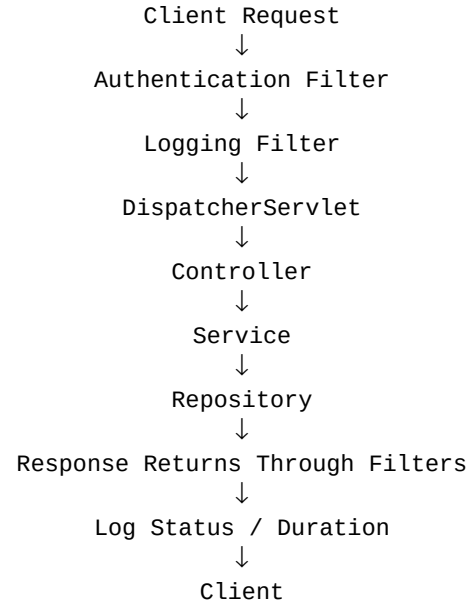
Why generate a request ID?

To correlate requests with logs and simplify troubleshooting.

Filter vs Interceptor?

Filter is part of the Servlet API and runs before DispatcherServlet; an interceptor is part of Spring MVC.

18. Final Revision Flow



One-line memory map

Final takeaway: Filters centralize common request/response operations—logging, authentication checks, request IDs and processing-time measurement—without repeating that logic in every controller.